

CI	Meaning
0b1	Critical error condition.

When clearing ERR<n>STATUS.V to 0, if this field is nonzero, then Arm recommends that software write 1 to this field to clear this field to zero.

The reset behavior of this field is:

- On a Cold reset, this field resets to an architecturally UNKNOWN value.

Accessing this field has the following behavior:

- When ERR<n>STATUS.V = '0', access to this field is UNKNOWN/WI.
- Otherwise, access to this field is W1C.

RV, bit [18]

When RAS System Architecture v2 is implemented:

Reset Valid. When ERR<n>STATUS.V is 1, indicating the error record is valid, this field indicates whether the error was recorded before or after the most recent Error Recovery reset.

RV	Meaning
0b0	If the error record is valid then one or more errors have been recorded after the last Error Recovery reset. This error or errors might have overwritten lower priority errors recorded before the last Error Recovery reset.
0b1	If the error record is valid then one or more errors were recorded before the last Error Recovery reset.

This field is set to 0 when an error is recorded and either the fault overwrites the error syndrome, or the error record was previously not valid.

The reset behavior of this field is:

- On a Error recovery reset, this field resets to '1'.

Access to this field is W1C.

Otherwise:

Reserved, RES0.

RV2, bit [17]

When RAS System Architecture v2 is implemented:

Reset Valid 2. When ERR<n>STATUS.{V, RV} is {1, 1}, indicating the error record is valid and one or more errors were recorded before the last Error Recovery reset, this field indicates whether any lower severity errors have been recorded after the Error Recovery reset that did not overwrite the syndrome.

RV2	Meaning
0b0	If the error record is valid then one or more errors were recorded after the last Error Recovery reset that did not overwrite the error syndrome. This includes errors that did not overwrite a previously recorded error syndrome.

RV2	Meaning
0b1	If the error record is valid then one or more errors were recorded before the last Error Recovery reset.

This field is set to 0 when an error is recorded, including when the fault does not overwrite a previously recorded syndrome.

The reset behavior of this field is:

- On a Error recovery reset, this field resets to '1'.

Access to this field is W1C.

Otherwise:

Reserved, RES0.

Bit [16]

Reserved, RES0.

IERR, bits [15:8]

IMPLEMENTATION DEFINED error code. Used with any primary error code ERR<n>STATUS.SERR value. Further IMPLEMENTATION DEFINED information can be placed in the ERR<n>MISC<m> registers.

The implemented set of valid values that this field can take is IMPLEMENTATION DEFINED. If any value not in this set is written to this register, then the value read back from this field is UNKNOWN.

Note

This means that one or more bits of this field might be implemented as fixed read-as-zero or read-as-one values.

The reset behavior of this field is:

- On a Cold reset, this field resets to an architecturally UNKNOWN value.

Accessing this field has the following behavior:

- Access to this field is UNKNOWN/WI if all of the following are true:
 - the node that owns error record n does not implement the Common Fault Injection Model Extension
 - ERR<n>STATUS.V == '0'
- Access to this field is UNKNOWN/WI if all of the following are true:
 - ERRPFGF[FirstRecordOfNode(n)].SYN == '0'
 - ERR<n>STATUS.V == '0'
- Otherwise, access to this field is RW.

SERR, bits [7:0]

Architecturally-defined primary error code. The primary error code might be used by a fault handling agent to triage an error without requiring device-specific code. For example, to count and threshold corrected errors in software, or generate a short log entry.

SERR	Meaning
0x00	No error.
0x01	IMPLEMENTATION DEFINED error.
0x02	Data value from (non-associative) internal memory. For example, ECC from on-chip SRAM or buffer.

SERR	Meaning
0x03	IMPLEMENTATION DEFINED pin. For example, nSEI pin.
0x04	Assertion failure. For example, consistency failure.
0x05	Error detected on internal data path. For example, parity on ALU result.
0x06	Data value from associative memory. For example, ECC error on cache data.
0x07	Address/control value from associative memory. For example, ECC error on cache tag.
0x08	Data value from a TLB. For example, ECC error on TLB data.
0x09	Address/control value from a TLB. For example, ECC error on TLB tag.
0x0A	Data value from producer. For example, parity error on write data bus.
0x0B	Address/control value from producer. For example, parity error on address bus.
0x0C	Data value from (non-associative) external memory. For example, ECC error in SDRAM.
0x0D	Illegal address (software fault). For example, access to unpopulated memory.
0x0E	Illegal access (software fault). For example, byte write to word register.
0x0F	Illegal state (software fault). For example, device not ready.
0x10	Internal data register. For example, parity on a SIMD&FP register. For a PE, all general-purpose, stack pointer, SIMD&FP, SVE, and SME registers are data registers.
0x11	Internal control register. For example, parity on a System register. For a PE, all registers other than general-purpose, stack pointer, SIMD&FP, SVE, and SME registers are control registers.
0x12	Error response from Completer of access. For example, error response from cache write-back.
0x13	External timeout. For example, timeout on interaction with another component.
0x14	Internal timeout. For example, timeout on interface within the component.
0x15	Deferred error from Completer not supported at Requester. For example, poisoned data received from the Completer of an access by a Requester that cannot defer the error further.
0x16	Deferred error from Requester not supported at Completer. For example, poisoned data received from the Requester of an access by a Completer that cannot defer the error further.
0x17	Deferred error from Completer passed through. For example, poisoned data received from the Completer of an access and returned to the Requester.
0x18	Deferred error from Requester passed through. For example, poisoned data received from the Requester of an access and deferred to the Completer.
0x19	Error recorded by PCIe error logs. Indicates that the component has recorded an error in a PCIe error log. This might be the PCIe device status register, AER, DVSEC, or other mechanisms defined by PCIe.
0x1A	Other internal error. For example, parity error on internal state of the component that is not covered by another primary error code.

All other values are reserved.

The implemented set of valid values that this field can take is IMPLEMENTATION DEFINED. If any value not in this set is written to this register, then the value read back from this field is UNKNOWN.

Note

This means that one or more bits of this field might be implemented as fixed read-as-zero or read-as-one values.

The reset behavior of this field is:

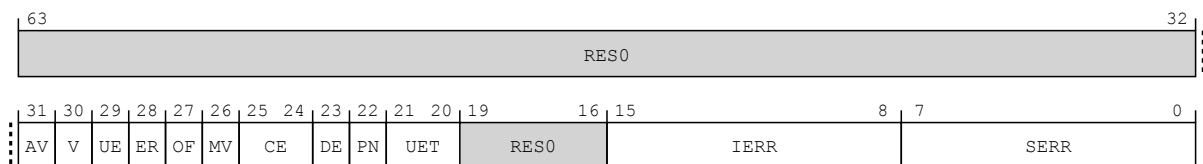
- On a Cold reset, this field resets to an architecturally UNKNOWN value.

Accessing this field has the following behavior:

- Access to this field is UNKNOWN/WI if all of the following are true:

- the node that owns error record n does not implement the Common Fault Injection Model Extension
- $ERR<n>STATUS.V == '0'$
- Access to this field is UNKNOWN/WI if all of the following are true:
 - $ERRPFGF[FirstRecordOfNode(n)].SYN == '0'$
 - $ERR<n>STATUS.V == '0'$
- Otherwise, access to this field is RW.

Otherwise:



Normal record, when FEAT_RASSAv1p1 is not implemented.

Bits [63:32]

Reserved, RES0.

AV, bit [31]

When error record n includes an address associated with an error:

Address Valid.

AV	Meaning
0b0	ERR<n>ADDR not valid.
0b1	ERR<n>ADDR contains an address associated with the highest priority error recorded by this record.

The reset behavior of this field is:

- On a Cold reset, this field resets to '0'.

Accessing this field has the following behavior:

- Access to this field is RO if all of the following are true:
 - $[ERR<n>STATUS.DE, ERR<n>STATUS.UE] == '00'$
 - $ERR<n>STATUS.CE != '00'$
 - $ERR<n>STATUS.CE$ is not being cleared to 0b00 in the same write
- Access to this field is RO if all of the following are true:
 - $ERR<n>STATUS.UE == '0'$
 - $ERR<n>STATUS.DE != '0'$
 - $ERR<n>STATUS.DE$ is not being cleared to 0b0 in the same write
- Access to this field is RO if all of the following are true:
 - $ERR<n>STATUS.UE != '0'$
 - $ERR<n>STATUS.UE$ is not being cleared to 0b0 in the same write
- Otherwise, access to this field is W1C.

Otherwise:

Reserved, RES0.

V, bit [30]

Status Register Valid.

V	Meaning
0b0	ERR<n>STATUS not valid.
0b1	ERR<n>STATUS valid. At least one error has been recorded.

The reset behavior of this field is:

- On a Cold reset, this field resets to '0'.

Accessing this field has the following behavior:

- Access to this field is RO if all of the following are true:
 - ERR<n>STATUS.CE != '00'
 - ERR<n>STATUS.CE is not being cleared to 0b00 in the same write
- Access to this field is RO if all of the following are true:
 - ERR<n>STATUS.DE != '0'
 - ERR<n>STATUS.DE is not being cleared to 0b0 in the same write
- Access to this field is RO if all of the following are true:
 - ERR<n>STATUS.UE != '0'
 - ERR<n>STATUS.UE is not being cleared to 0b0 in the same write
- Otherwise, access to this field is W1C.

UE, bit [29]

Uncorrected Error.

UE	Meaning
0b0	No errors have been detected, or all detected errors have been either corrected or deferred.
0b1	At least one detected error was not corrected and not deferred.

When clearing ERR<n>STATUS.V to 0, if this field is nonzero, then Arm recommends that software write 1 to this field to clear this field to zero.

The reset behavior of this field is:

- On a Cold reset, this field resets to an architecturally UNKNOWN value.

Accessing this field has the following behavior:

- When ERR<n>STATUS.V == '0', access to this field is UNKNOWN/WI.
- Access to this field is RO if all of the following are true:
 - ERR<n>STATUS.OF == '1'
 - ERR<n>STATUS.OF is not being cleared to 0b0 in the same write
- Otherwise, access to this field is W1C.

ER, bit [28]

When in-band error responses can be returned for a Deferred error:

Error Reported.

ER	Meaning
0b0	No in-band error response (External abort) signaled to the Requester making the access or other transaction.
0b1	An in-band error response was signaled by the component to the Requester making the access or other transaction. This can be because any of the following are true: - The ERRCTL[FirstRecordOfNode(n)].UE field, or applicable one of the ERRCTL[FirstRecordOfNode(n)].{WUE, RUE} fields, is implemented and was 1 when an error was detected and not corrected. - The ERRCTL[FirstRecordOfNode(n)].{WUE, RUE, UE} fields are not implemented and the component always reports errors.

If this field is nonzero, then Arm recommends that software write 1 to this field to clear this field to zero, when any of:

- Clearing `ERR<n>STATUS.V` to 0.
- Clearing both `ERR<n>STATUS.{DE, UE}` to 0.

The reset behavior of this field is:

- On a Cold reset, this field resets to an architecturally UNKNOWN value.

Accessing this field has the following behavior:

- Access to this field is UNKNOWN/WI if any of the following are true:
 - `ERR<n>STATUS.V == '0'`
 - `[ERR<n>STATUS.DE, ERR<n>STATUS.UE] == '00'`
- Access to this field is RO if all of the following are true:
 - `ERR<n>STATUS.UE != '0'`
 - `ERR<n>STATUS.UE` is not being cleared to 0b0 in the same write
- Access to this field is RO if all of the following are true:
 - `ERR<n>STATUS.UE == '0'`
 - `ERR<n>STATUS.DE != '0'`
 - `ERR<n>STATUS.DE` is not being cleared to 0b0 in the same write
- Otherwise, access to this field is WIC.

When in-band error responses are never returned for a Deferred error:

Error Reported.

ER	Meaning
0b0	No in-band error response (External abort) signaled to the Requester making the access or other transaction.
0b1	An in-band error response was signaled by the component to the Requester making the access or other transaction. This can be because any of the following are true: - The ERRCTL[FirstRecordOfNode(n)].UE field, or applicable one of the ERRCTL[FirstRecordOfNode(n)].{WUE, RUE} fields, is implemented and was 1 when an error was detected and not corrected. - The ERRCTL[FirstRecordOfNode(n)].{WUE, RUE, UE} fields are not implemented and the component always reports errors.

If this field is nonzero, then Arm recommends that software write 1 to this field to clear this field to zero, when any of:

- Clearing ERR<n>STATUS.V to 0.
- Clearing ERR<n>STATUS.UE to 0.

The reset behavior of this field is:

- On a Cold reset, this field resets to an architecturally UNKNOWN value.

Accessing this field has the following behavior:

- Access to this field is UNKNOWN/WI if any of the following are true:
 - ERR<n>STATUS.V == '0'
 - ERR<n>STATUS.UE == '0'
- Access to this field is RO if all of the following are true:
 - ERR<n>STATUS.UE != '0'
 - ERR<n>STATUS.UE is not being cleared to 0b0 in the same write
- Otherwise, access to this field is WIC.

Otherwise:

Reserved, RES0.

OF, bit [27]

Overflow.

Indicates that multiple errors have been detected. This field is set to 1 when one of the following occurs:

- An Uncorrected error is detected and ERR<n>STATUS.UE == 1.
- A Deferred error is detected, ERR<n>STATUS.UE == 0 and ERR<n>STATUS.DE == 1.
- A Corrected error is detected, no Corrected error counter is implemented, ERR<n>STATUS.UE == 0, ERR<n>STATUS.DE == 0, and ERR<n>STATUS.CE != 0b00. ERR<n>STATUS.CE might be updated for the new Corrected error.
- A Corrected error counter is implemented, ERR<n>STATUS.UE == 0, ERR<n>STATUS.DE == 0, and the counter overflows.

It is IMPLEMENTATION DEFINED whether this field is set to 1 when one of the following occurs:

- A Deferred error is detected and ERR<n>STATUS.UE == 1.
- A Corrected error is detected, no Corrected error counter is implemented, and ERR<n>STATUS.{UE, DE} != {0, 0}.
- A Corrected error counter is implemented, ERR<n>STATUS.{UE, DE} != {0, 0}, and the counter overflows.

It is IMPLEMENTATION DEFINED whether this field is cleared to 0 when one of the following occurs:

- An Uncorrected error is detected and ERR<n>STATUS.UE == 0.
- A Deferred error is detected, ERR<n>STATUS.UE == 0, and ERR<n>STATUS.DE == 0.
- A Corrected error is detected, ERR<n>STATUS.UE == 0, ERR<n>STATUS.DE == 0, and ERR<n>STATUS.CE == 0b00.

The IMPLEMENTATION DEFINED clearing of this field might also depend on the value of the other error status fields.

If a Corrected error counter is implemented, then:

- A direct write that modifies the counter overflow flag indirectly might set this field to an UNKNOWN value.
- A direct write to this field that clears this field to 0 might indirectly set the counter overflow flag to an UNKNOWN value.

OF	Meaning
0b0	If $\text{ERR}<n>\text{STATUS.UE} == 1$, then no error syndrome for an Uncorrected error has been discarded. If $\text{ERR}<n>\text{STATUS.UE} == 0$ and $\text{ERR}<n>\text{STATUS.DE} == 1$, then no error syndrome for a Deferred error has been discarded. If $\text{ERR}<n>\text{STATUS.UE} == 0$, $\text{ERR}<n>\text{STATUS.DE} == 0$, and a Corrected error counter is implemented, then the counter has not overflowed. If $\text{ERR}<n>\text{STATUS.UE} == 0$, $\text{ERR}<n>\text{STATUS.DE} == 0$, $\text{ERR}<n>\text{STATUS.CE} != 0b00$, and no Corrected error counter is implemented, then no error syndrome for a Corrected error has been discarded. Note This field might have been set to 1 when an error syndrome was discarded and later cleared to 0 when a higher priority syndrome was recorded.
0b1	At least one error syndrome has been discarded or, if a Corrected error counter is implemented, it might have overflowed.

When clearing $\text{ERR}<n>\text{STATUS.V}$ to 0, if this field is nonzero, then Arm recommends that software write 1 to this field to clear this field to zero.

The reset behavior of this field is:

- On a Cold reset, this field resets to an architecturally UNKNOWN value.

Accessing this field has the following behavior:

- When $\text{ERR}<n>\text{STATUS.V} == '0'$, access to this field is UNKNOWN/WI.
- Otherwise, access to this field is WIC.

MV, bit [26]

When error record <n> includes additional information for an error:

Miscellaneous Registers Valid.

MV	Meaning
0b0	$\text{ERR}<n>\text{MISC}<m>$ not valid.
0b1	The contents of the $\text{ERR}<n>\text{MISC}<m>$ registers contain additional information for an error recorded by this record.

Note

If the $\text{ERR}<n>\text{MISC}<m>$ registers can contain additional information for a previously recorded error, then the contents must be self-describing to software or a user. For example, certain fields might relate only to Corrected errors, and other fields only to the most recent error that was not discarded.

The reset behavior of this field is:

- On a Cold reset, this field resets to '0'.

Accessing this field has the following behavior:

- Access to this field is RO if all of the following are true:
 - $[\text{ERR}<n>\text{STATUS.DE}, \text{ERR}<n>\text{STATUS.UE}] == '00'$
 - $\text{ERR}<n>\text{STATUS.CE} != '00'$
 - $\text{ERR}<n>\text{STATUS.CE}$ is not being cleared to 0b00 in the same write
- Access to this field is RO if all of the following are true:
 - $\text{ERR}<n>\text{STATUS.UE} == '0'$

- $ERR\langle n \rangle STATUS.DE \neq '0'$
- $ERR\langle n \rangle STATUS.DE$ is not being cleared to 0b0 in the same write
- Access to this field is RO if all of the following are true:
 - $ERR\langle n \rangle STATUS.UE \neq '0'$
 - $ERR\langle n \rangle STATUS.UE$ is not being cleared to 0b0 in the same write
- Otherwise, access to this field is W1C.

Otherwise:

Reserved, RES0.

CE, bits [25:24]

Corrected Error.

CE	Meaning
0b00	No errors were corrected.
0b01	At least one transient error was corrected.
0b10	At least one error was corrected.
0b11	At least one persistent error was corrected.

The mechanism by which a component or node detects whether a Corrected error is transient or persistent is IMPLEMENTATION DEFINED. If no such mechanism is implemented, then the node sets this field to 0b10 when a corrected error is recorded.

When clearing $ERR\langle n \rangle STATUS.V$ to 0, if this field is nonzero, then Arm recommends that software write ones to this field to clear this field to zero.

The reset behavior of this field is:

- On a Cold reset, this field resets to an architecturally UNKNOWN value.

Accessing this field has the following behavior:

- When $ERR\langle n \rangle STATUS.V = '0'$, access to this field is UNKNOWN/WI.
- Access to this field is RO if all of the following are true:
 - $ERR\langle n \rangle STATUS.OF = '1'$
 - $ERR\langle n \rangle STATUS.OF$ is not being cleared to 0b0 in the same write
- Otherwise, access to this field is W1C.

DE, bit [23]

Deferred Error.

DE	Meaning
0b0	No errors were deferred.
0b1	At least one error was not corrected and deferred.

Support for deferring errors is IMPLEMENTATION DEFINED.

When clearing $ERR\langle n \rangle STATUS.V$ to 0, if this field is nonzero, then Arm recommends that software write 1 to this field to clear this field to zero.

The reset behavior of this field is:

- On a Cold reset, this field resets to an architecturally UNKNOWN value.

Accessing this field has the following behavior:

- When $ERR<n>STATUS.V = '0'$, access to this field is UNKNOWN/WI.
- Access to this field is RO if all of the following are true:
 - $ERR<n>STATUS.OF = '1'$
 - $ERR<n>STATUS.OF$ is not being cleared to 0b0 in the same write
- Otherwise, access to this field is WIC.

PN, bit [22]

Poison.

PN	Meaning
0b0	Uncorrected error or Deferred error recorded because a corrupt value was detected, for example, by an error detection code (EDC), or Corrected error recorded.
0b1	Uncorrected error or Deferred error recorded because a poison value was detected.

If this field is nonzero, then Arm recommends that software write 1 to this field to clear this field to zero, when any of:

- Clearing $ERR<n>STATUS.V$ to 0.
- Clearing both $ERR<n>STATUS.\{DE, UE\}$ to 0.

The reset behavior of this field is:

- On a Cold reset, this field resets to an architecturally UNKNOWN value.

Accessing this field has the following behavior:

- Access to this field is UNKNOWN/WI if any of the following are true:
 - $ERR<n>STATUS.V = '0'$
 - $[ERR<n>STATUS.DE, ERR<n>STATUS.UE] = '00'$
- Access to this field is RO if all of the following are true:
 - $[ERR<n>STATUS.DE, ERR<n>STATUS.UE] = '00'$
 - $ERR<n>STATUS.CE \neq '00'$
 - $ERR<n>STATUS.CE$ is not being cleared to 0b00 in the same write
- Access to this field is RO if all of the following are true:
 - $ERR<n>STATUS.UE = '0'$
 - $ERR<n>STATUS.DE \neq '0'$
 - $ERR<n>STATUS.DE$ is not being cleared to 0b0 in the same write
- Access to this field is RO if all of the following are true:
 - $ERR<n>STATUS.UE \neq '0'$
 - $ERR<n>STATUS.UE$ is not being cleared to 0b0 in the same write
- Otherwise, access to this field is WIC.

UET, bits [21:20]

Uncorrected Error Type. Describes the state of the component after detecting or consuming an Uncorrected error.

UET	Meaning
0b00	Uncorrected error, Uncontainable error (UC).

UET	Meaning
0b01	Uncorrected error, Unrecoverable error (UEU).
0b10	Uncorrected error, Latent or Restartable error (UEO).
0b11	Uncorrected error, Signaled or Recoverable error (UER).

UER can mean either Signaled or Recoverable error, and UEO can mean either Latent or Restartable error.

If this field is nonzero, then Arm recommends that software write ones to this field to clear this field to zero, when any of:

- Clearing ERR<n>STATUS.V to 0.
- Clearing ERR<n>STATUS.UE to 0.

The reset behavior of this field is:

- On a Cold reset, this field resets to an architecturally UNKNOWN value.

Accessing this field has the following behavior:

- Access to this field is UNKNOWN/WI if any of the following are true:
 - ERR<n>STATUS.V == '0'
 - ERR<n>STATUS.UE == '0'
- Access to this field is RO if all of the following are true:
 - [ERR<n>STATUS.DE, ERR<n>STATUS.UE] == '00'
 - ERR<n>STATUS.CE != '00'
 - ERR<n>STATUS.CE is not being cleared to 0b00 in the same write
- Access to this field is RO if all of the following are true:
 - ERR<n>STATUS.UE == '0'
 - ERR<n>STATUS.DE != '0'
 - ERR<n>STATUS.DE is not being cleared to 0b0 in the same write
- Access to this field is RO if all of the following are true:
 - ERR<n>STATUS.UE != '0'
 - ERR<n>STATUS.UE is not being cleared to 0b0 in the same write
- Otherwise, access to this field is W1C.

Bits [19:16]

Reserved, RES0.

IERR, bits [15:8]

IMPLEMENTATION DEFINED error code. Used with any primary error code ERR<n>STATUS.SERR value. Further IMPLEMENTATION DEFINED information can be placed in the ERR<n>MISC<m> registers.

The implemented set of valid values that this field can take is IMPLEMENTATION DEFINED. If any value not in this set is written to this register, then the value read back from this field is UNKNOWN.

Note

This means that one or more bits of this field might be implemented as fixed read-as-zero or read-as-one values.

The reset behavior of this field is:

- On a Cold reset, this field resets to an architecturally UNKNOWN value.

Accessing this field has the following behavior:

- Access to this field is UNKNOWN/WI if all of the following are true:
 - the node that owns error record n does not implement the Common Fault Injection Model Extension
 - ERR<n>STATUS.V == '0'
- Access to this field is UNKNOWN/WI if all of the following are true:

- $ERRPFGF[FirstRecordOfNode(n)].SYN == '0'$
- $ERR<n>STATUS.V == '0'$
- Access to this field is RO if all of the following are true:
 - $[ERR<n>STATUS.DE, ERR<n>STATUS.UE] == '00'$
 - $ERR<n>STATUS.CE != '00'$
 - $ERR<n>STATUS.CE$ is not being cleared to 0b00 in the same write
- Access to this field is RO if all of the following are true:
 - $ERR<n>STATUS.UE == '0'$
 - $ERR<n>STATUS.DE != '0'$
 - $ERR<n>STATUS.DE$ is not being cleared to 0b0 in the same write
- Access to this field is RO if all of the following are true:
 - $ERR<n>STATUS.UE != '0'$
 - $ERR<n>STATUS.UE$ is not being cleared to 0b0 in the same write
- Otherwise, access to this field is RW.

SERR, bits [7:0]

Architecturally-defined primary error code. The primary error code might be used by a fault handling agent to triage an error without requiring device-specific code. For example, to count and threshold corrected errors in software, or generate a short log entry.

SERR	Meaning
0x00	No error.
0x01	IMPLEMENTATION DEFINED error.
0x02	Data value from (non-associative) internal memory. For example, ECC from on-chip SRAM or buffer.
0x03	IMPLEMENTATION DEFINED pin. For example, nSEI pin.
0x04	Assertion failure. For example, consistency failure.
0x05	Error detected on internal data path. For example, parity on ALU result.
0x06	Data value from associative memory. For example, ECC error on cache data.
0x07	Address/control value from associative memory. For example, ECC error on cache tag.
0x08	Data value from a TLB. For example, ECC error on TLB data.
0x09	Address/control value from a TLB. For example, ECC error on TLB tag.
0x0A	Data value from producer. For example, parity error on write data bus.
0x0B	Address/control value from producer. For example, parity error on address bus.
0x0C	Data value from (non-associative) external memory. For example, ECC error in SDRAM.
0x0D	Illegal address (software fault). For example, access to unpopulated memory.
0x0E	Illegal access (software fault). For example, byte write to word register.
0x0F	Illegal state (software fault). For example, device not ready.
0x10	Internal data register. For example, parity on a SIMD&FP register. For a PE, all general-purpose, stack pointer, SIMD&FP, SVE, and SME registers are data registers.
0x11	Internal control register. For example, parity on a System register. For a PE, all registers other than general-purpose, stack pointer, SIMD&FP, SVE, and SME registers are control registers.
0x12	Error response from Completer of access. For example, error response from cache write-back.
0x13	External timeout. For example, timeout on interaction with another component.
0x14	Internal timeout. For example, timeout on interface within the component.

SERR	Meaning
0x15	Deferred error from Completer not supported at Requester. For example, poisoned data received from the Completer of an access by a Requester that cannot defer the error further.
0x16	Deferred error from Requester not supported at Completer. For example, poisoned data received from the Requester of an access by a Completer that cannot defer the error further.
0x17	Deferred error from Completer passed through. For example, poisoned data received from the Completer of an access and returned to the Requester.
0x18	Deferred error from Requester passed through. For example, poisoned data received from the Requester of an access and deferred to the Completer.
0x19	Error recorded by PCIe error logs. Indicates that the component has recorded an error in a PCIe error log. This might be the PCIe device status register, AER, DVSEC, or other mechanisms defined by PCIe.
0x1A	Other internal error. For example, parity error on internal state of the component that is not covered by another primary error code.

All other values are reserved.

The implemented set of valid values that this field can take is IMPLEMENTATION DEFINED. If any value not in this set is written to this register, then the value read back from this field is UNKNOWN.

Note

This means that one or more bits of this field might be implemented as fixed read-as-zero or read-as-one values.

The reset behavior of this field is:

- On a Cold reset, this field resets to an architecturally UNKNOWN value.

Accessing this field has the following behavior:

- Access to this field is UNKNOWN/WI if all of the following are true:
 - the node that owns error record n does not implement the Common Fault Injection Model Extension
 - ERR<n>STATUS.V == '0'
- Access to this field is UNKNOWN/WI if all of the following are true:
 - ERRPFGF[FirstRecordOfNode(n)].SYN == '0'
 - ERR<n>STATUS.V == '0'
- Access to this field is RO if all of the following are true:
 - ERR<n>STATUS.DE == '0'
 - ERR<n>STATUS.UE == '0'
 - ERR<n>STATUS.CE != '00'
 - ERR<n>STATUS.CE is not being cleared to 0b00 in the same write
- Access to this field is RO if all of the following are true:
 - ERR<n>STATUS.UE == '0'
 - ERR<n>STATUS.DE != '0'
 - ERR<n>STATUS.DE is not being cleared to 0b0 in the same write
- Access to this field is RO if all of the following are true:
 - ERR<n>STATUS.UE != '0'
 - ERR<n>STATUS.UE is not being cleared to 0b0 in the same write
- Otherwise, access to this field is RW.

Accessing ERR<n>STATUS

ERR<n>STATUS.{AV, V, UE, ER, OF, MV, CE, DE, PN, UET, CI} are write-one-to-clear (W1C) fields, meaning writes of zero are ignored, and a write of one or all-ones to the field clears the field to zero. ERR<n>STATUS.{IERR, SERR} are read/write (RW) fields, although the set of implemented valid values is IMPLEMENTATION DEFINED. See also [ERR<n>PFGF.SYN](#).

After reading ERR<n>STATUS, software must clear the valid fields in the register to allow new errors to be recorded. However, between reading the register and clearing the valid fields, a new error might have overwritten the register. To

prevent this error being lost by software, the register prevents updates to fields that might have been updated by a new error.

When RAS System Architecture v1.0 is implemented:

- Writes to ERR<n>STATUS.{UE, DE, CE} are ignored if ERR<n>STATUS.OF is 1 and is not being cleared to 0.
- Writes to ERR<n>STATUS.V are ignored if any of ERR<n>STATUS.{UE, DE, CE} are nonzero and are not being cleared to zero.
- Writes to ERR<n>STATUS.{AV, MV} and the ERR<n>STATUS.{ER, PN, UET, IERR, SERR} syndrome fields are ignored if the highest priority nonzero error status field is not being cleared to zero. The error status fields in priority order from highest to lowest, are ERR<n>STATUS.UE, ERR<n>STATUS.DE, and ERR<n>STATUS.CE.

When RAS System Architecture v1.1 is implemented, a write to the register is ignored if all of:

- Any of ERR<n>STATUS.{V, UE, OF, CE, DE} are nonzero before the write.
- The write does not clear the nonzero ERR<n>STATUS.{V, UE, OF, CE, DE} fields to zero by writing ones to the applicable field or fields.

Some of the fields in ERR<n>STATUS are also defined as UNKNOWN where certain combinations of ERR<n>STATUS.{V, DE, UE} are zero. The rules for writes to ERR<n>STATUS allow a node to implement such a field as a fixed read-only value.

For example, when RAS System Architecture v1.1 is implemented, a write to ERR<n>STATUS when ERR<n>STATUS.V is 1 results in either ERR<n>STATUS.V field being cleared to zero, or ERR<n>STATUS.V not changing. Since all fields in ERR<n>STATUS, other than ERR<n>STATUS.{AV, V, MV}, usually read as UNKNOWN values when ERR<n>STATUS.V is zero, this means those fields can be implemented as read-only if applicable.

To ensure correct and portable operation, when software is clearing the valid fields in the register to allow new errors to be recorded, Arm recommends that software performs the following sequence of operations in order:

1. Read ERR<n>STATUS and determine which fields need to be cleared to zero.
2. In a single write to ERR<n>STATUS:
 - Write ones to all the W1C fields that are nonzero in the read value.
 - Write zero to all the W1C fields that are zero in the read value.
 - Write zero to all the RW fields.
3. Read back ERR<n>STATUS after the write to confirm no new fault has been recorded.

Otherwise, these fields might not have the correct value when a new fault is recorded.

ERR<n>STATUS can be accessed through the memory-mapped interface:

Component	Offset	Instance
RAS	$0x010 + (64 * n)$	ERR<n>STATUS

Accessible as follows:

- When ERR<n>STATUS.V != '0', ERR<n>STATUS.V is not being cleared to 0b0 in the same write, and RAS System Architecture v1p1 is implemented, accesses to this register are RO.
- When ERR<n>STATUS.UE != '0', ERR<n>STATUS.UE is not being cleared to 0b0 in the same write, and RAS System Architecture v1p1 is implemented, accesses to this register are RO.
- When ERR<n>STATUS.OF != '0', ERR<n>STATUS.OF is not being cleared to 0b0 in the same write, and RAS System Architecture v1p1 is implemented, accesses to this register are RO.
- When ERR<n>STATUS.CE != '00', ERR<n>STATUS.CE is not being cleared to 0b00 in the same write, and RAS System Architecture v1p1 is implemented, accesses to this register are RO.
- When ERR<n>STATUS.DE != '0', ERR<n>STATUS.DE is not being cleared to 0b0 in the same write, and RAS System Architecture v1p1 is implemented, accesses to this register are RO.
- Otherwise, accesses to this register are RW.

3.2.34 ERRPIDR0, Peripheral Identification Register 0

The ERRPIDR0 characteristics are:

Purpose

Provides discovery information about the component.

Configuration

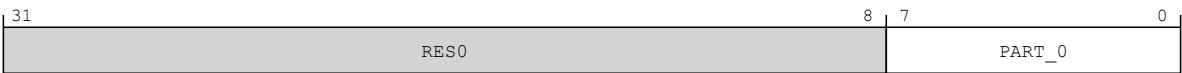
ERRPIDR0 is implemented only as part of a memory-mapped group of error records.

Implementation of this register is OPTIONAL.

Attributes

ERRPIDR0 is a 32-bit register.

Field descriptions



Bits [31:8]

Reserved, RES0.

PART_0, bits [7:0]

Part number, which is selected by the designer of the component and stored as follows:

- For a component with a 12-bit part number:
 - [ERRPIDR1](#).PART_1 contains part number bits [11:8].
 - [ERRPIDR0](#).PART_0 contains part number bits [7:0].
- For a component with a 16-bit part number:
 - [ERRPIDR1](#).PART_1 contains part number bits [15:12].
 - [ERRPIDR0](#).PART_0 contains part number bits [11:4].
 - [ERRPIDR2](#).REVISION contains part number bits [3:0].

When a 12-bit part number is used, [ERRPIDR2](#).REVISION indicates revision information.

The choice of using a 12-bit part number or 16-bit part number is specific to the designer of the component.

This field has an IMPLEMENTATION DEFINED value.

Access to this field is RO.

Accessing ERRPIDR0

This section shows the offset of ERRPIDR0 when FEAT_RASSA_4KB_GRP is implemented. If FEAT_RASSA_16KB_GRP or FEAT_RASSA_64KB_GRP is implemented, see ‘RAS memory-mapped register views’ for the offset of ERRPIDR0.

ERRPIDR0 can be accessed through the memory-mapped interface:

Component	Offset	Instance
RAS	0xFE0	ERRPIDR0

Accesses to this register are RO.

3.2.35 ERRPIDR1, Peripheral Identification Register 1

The ERRPIDR1 characteristics are:

Purpose

Provides discovery information about the component.

Configuration

ERRPIDR1 is implemented only as part of a memory-mapped group of error records.

Implementation of this register is OPTIONAL.

Attributes

ERRPIDR1 is a 32-bit register.

Field descriptions



Bits [31:8]

Reserved, RES0.

DES_0, bits [7:4]

Designer, JEP106 identification code, bits [3:0].

The JEP106 identification and continuation codes are stored as follows:

- [ERRPIDR1.DES_0](#): JEP106 identification code bits[3:0].
- [ERRPIDR2.DES_1](#): JEP106 identification code bits[6:4].
- [ERRPIDR4.DES_2](#): JEP106 continuation code.

These codes indicate the designer of the component and not the implementer, except where the two are the same. To obtain a number, or to see the assignment of these codes, contact JEDEC <http://www.jedec.org>.

A JEP106 identification and continuation code takes the following form:

- A sequence of zero or more numbers, all having the value $0 \times 7F$.
- A following 8-bit number, that is not $0 \times 7F$, and where bit[7] is an odd parity bit.

The parity bit in the JEP106 identification code is not included.

This field has an IMPLEMENTATION DEFINED value.

Note

For example, Arm Limited is assigned the code $0 \times 7F \ 0 \times 7F \ 0 \times 7F \ 0 \times 7F \ 0 \times 3B$.

- The continuation code is the number of times $0 \times 7F$ appears before the final number. For example, a component designed by Arm Limited has the code 0×4 .
- The identification code is bits[6:0] of the final number. For example, a component designed by Arm Limited has the code $0 \times 3B$.

Access to this field is RO.

PART_1, bits [3:0]

Part number, which is selected by the designer of the component and stored as follows:

- For a component with a 12-bit part number:
 - [ERRPIDR1](#).PART_1 contains part number bits [11:8].
 - [ERRPIDR0](#).PART_0 contains part number bits [7:0].
- For a component with a 16-bit part number:
 - [ERRPIDR1](#).PART_1 contains part number bits [15:12].
 - [ERRPIDR0](#).PART_0 contains part number bits [11:4].
 - [ERRPIDR2](#).REVISION contains part number bits [3:0].

When a 12-bit part number is used, [ERRPIDR2](#).REVISION indicates revision information.

The choice of using a 12-bit part number or 16-bit part number is specific to the designer of the component.

This field has an IMPLEMENTATION DEFINED value.

Access to this field is RO.

Accessing ERRPIDR1

This section shows the offset of ERRPIDR1 when FEAT_RASSA_4KB_GRP is implemented. If FEAT_RASSA_16KB_GRP or FEAT_RASSA_64KB_GRP is implemented, see 'RAS memory-mapped register views' for the offset of ERRPIDR1.

ERRPIDR1 can be accessed through the memory-mapped interface:

Component	Offset	Instance
RAS	0xFE4	ERRPIDR1

Accesses to this register are RO.

3.2.36 ERRPIDR2, Peripheral Identification Register 2

The ERRPIDR2 characteristics are:

Purpose

Provides discovery information about the component.

Configuration

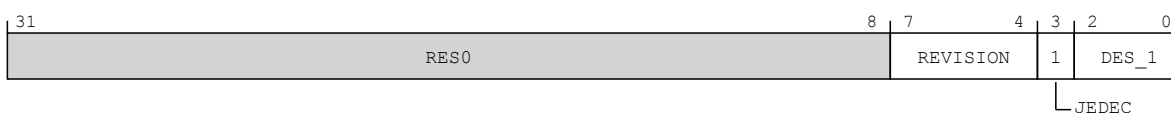
ERRPIDR2 is implemented only as part of a memory-mapped group of error records.

Implementation of this register is OPTIONAL.

Attributes

ERRPIDR2 is a 32-bit register.

Field descriptions



Bits [31:8]

Reserved, RES0.

REVISION, bits [7:4]

Indicates either the revision of the component, or a portion of the part number of the component.

Where the component has a single 4-bit revision number, the revision number is an incremental value starting at zero for the first revision of the component.

Where the component has separate major and minor revision numbers, the major and minor revision numbers are each incremental values starting at zero for the first revision of the component. For each minor revision of the component, the minor revision number increments monotonically. For each major revision of the component, the major revision number increments monotonically and the minor revision begins again at zero.

For a component with a 12-bit part number with a single 4-bit revision number:

- [ERRPIDR2.REVISION](#) indicates the 4-bit revision number.
- [ERRPIDR3.REVAND](#) indicates component modifications.

For a component with a 12-bit part number with separate major and minor revision numbers:

- [ERRPIDR2.REVISION](#) indicates the 4-bit major revision number.
- [ERRPIDR3.REVAND](#) indicates the 4-bit minor revision number.

For a component with a 16-bit part number:

- [ERRPIDR2.REVISION](#) contains part number bits [3:0].
- [ERRPIDR3.REVAND](#) indicates the 4-bit revision number.

The choice of which style of revision information is used is specific to the designer of the component, and might also be specific to each individual component with a different part number.

This field has an IMPLEMENTATION DEFINED value.

Previous versions of this specification named this field PART_2 when the component uses a 16-bit part number.

Access to this field is RO.

JEDEC, bit [3]

JEDEC-assigned JEP106 implementer code is used.

Reads as 0b1

Access to this field is RO.

DES_1, bits [2:0]

Designer, JEP106 identification code, bits [6:4].

The JEP106 identification and continuation codes are stored as follows:

- [ERRPIDR1.DES_0](#): JEP106 identification code bits[3:0].
- [ERRPIDR2.DES_1](#): JEP106 identification code bits[6:4].
- [ERRPIDR4.DES_2](#): JEP106 continuation code.

These codes indicate the designer of the component and not the implementer, except where the two are the same. To obtain a number, or to see the assignment of these codes, contact JEDEC <http://www.jedec.org>.

A JEP106 identification and continuation code takes the following form:

- A sequence of zero or more numbers, all having the value 0x7F.
- A following 8-bit number, that is not 0x7F, and where bit[7] is an odd parity bit.

The parity bit in the JEP106 identification code is not included.

This field has an IMPLEMENTATION DEFINED value.

Note

For example, Arm Limited is assigned the code 0x7F 0x7F 0x7F 0x7F 0x3B.

- The continuation code is the number of times 0x7F appears before the final number. For example, a component designed by Arm Limited has the code 0x4.
 - The identification code is bits[6:0] of the final number. For example, a component designed by Arm Limited has the code 0x3B.
-

Access to this field is RO.

Accessing ERRPIDR2

This section shows the offset of ERRPIDR2 when FEAT_RASSA_4KB_GRP is implemented. If FEAT_RASSA_16KB_GRP or FEAT_RASSA_64KB_GRP is implemented, see ‘RAS memory-mapped register views’ for the offset of ERRPIDR2.

ERRPIDR2 can be accessed through the memory-mapped interface:

Component	Offset	Instance
RAS	0xFE8	ERRPIDR2

Accesses to this register are RO.

3.2.37 ERRPIDR3, Peripheral Identification Register 3

The ERRPIDR3 characteristics are:

Purpose

Provides discovery information about the component.

Configuration

ERRPIDR3 is implemented only as part of a memory-mapped group of error records.

Implementation of this register is **OPTIONAL**.

Attributes

ERRPIDR3 is a 32-bit register.

Field descriptions

31																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																															
----	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--

Bits [31:8]

Reserved, RES0.

REVAND, bits [7:4]

Indicates either the revision of the component, or whether the component has been modified.

Where the component has a single 4-bit revision number, the revision number is an incremental value starting at zero for the first revision of the component.

Where the component has separate major and minor revision numbers, the major and minor revision numbers are each incremental values starting at zero for the first revision of the component. For each minor revision of the component, the minor revision number increments monotonically. For each major revision of the component, the major revision number increments monotonically and the minor revision begins again at zero.

For a component with a 12-bit part number with a single 4-bit revision number:

- [ERRPIDR2.REVISION](#) indicates the 4-bit revision number.
- [ERRPIDR3.REVAND](#) indicates component modifications.

For a component with a 12-bit part number with separate major and minor revision numbers:

- [ERRPIDR2.REVISION](#) indicates the 4-bit major revision number.
- [ERRPIDR3.REVAND](#) indicates the 4-bit minor revision number.

For a component with a 16-bit part number:

- [ERRPIDR2.REVISION](#) contains part number bits [3:0].
- [ERRPIDR3.REVAND](#) indicates the 4-bit revision number.

The choice of which style of revision information is used is specific to the designer of the component, and might also be specific to each individual component with a different part number.

This field has an **IMPLEMENTATION DEFINED** value.

Where REVAND indicates component modifications, this indicates modifications such as errata fixes or metal fixes after implementation. Usually this value would be zero unless a modification has been performed. If the field is required for indicating component modifications, Arm recommends that component designers ensure that it can be changed by a metal fix, for example by driving it from registers that reset to zero.

[ERRPIDR3](#).CMOD might also indicate component modifications.

Previous versions of this specification named this field REVISION when the component uses a 16-bit part number.

Access to this field is RO.

CMOD, bits [3:0]

Indicates whether the component has been modified from its original behavior. Examples of modifications include errata fixes or metal fixes after implementation. Usually this value would be zero unless a modification has been performed. If the field is required for indicating component modifications, Arm recommends that component designers ensure that it can be changed by a metal fix, for example by driving it from registers that reset to zero.

A value of 0b0000 means the component is not modified from the original design.

Any other value means the component has been modified in an IMPLEMENTATION DEFINED way.

This field has an IMPLEMENTATION DEFINED value.

For any two components with the same Unique Component Identifier:

- If the value of the CMOD fields of both components equals zero, the components are identical.
- If the CMOD fields of both components have the same nonzero value, it does not necessarily mean that they have the same modifications.
- If the value of the CMOD field of either of the two components is nonzero, they might not be identical, even though they have the same Unique Component Identifier.

[ERRPIDR3](#).REVAND might also indicate component modifications.

Access to this field is RO.

Accessing ERRPIDR3

This section shows the offset of ERRPIDR3 when FEAT_RASSA_4KB_GRP is implemented. If FEAT_RASSA_16KB_GRP or FEAT_RASSA_64KB_GRP is implemented, see ‘RAS memory-mapped register views’ for the offset of ERRPIDR3.

ERRPIDR3 can be accessed through the memory-mapped interface:

Component	Offset	Instance
RAS	0xFEC	ERRPIDR3

Accesses to this register are RO.

3.2.38 ERRPIDR4, Peripheral Identification Register 4

The ERRPIDR4 characteristics are:

Purpose

Provides discovery information about the component.

Configuration

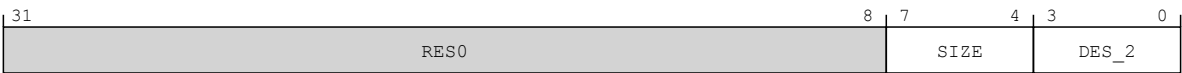
ERRPIDR4 is implemented only as part of a memory-mapped group of error records.

Implementation of this register is OPTIONAL.

Attributes

ERRPIDR4 is a 32-bit register.

Field descriptions



Bits [31:8]

Reserved, RES0.

SIZE, bits [7:4]

When RAS System Architecture v2 is implemented:

Size of the component.

SIZE	Meaning
0b0000	FEAT_RASSA_4KB is implemented.
0b0010	FEAT_RASSA_16KB is implemented.
0b0100	FEAT_RASSA_64KB is implemented.

All other values are reserved.

Otherwise:

Size of the component.

SIZE	Meaning
0b0000	One of the following is true: - The component uses a single 4KB block. - The component uses an IMPLEMENTATION DEFINED number of 4KB blocks.

SIZE	Meaning
0b0001..0b1111	The component occupies $2^{\text{ERRPIDR4.SIZE}}$ 4KB blocks.

DES_2, bits [3:0]

Designer, JEP106 continuation code.

The JEP106 identification and continuation codes are stored as follows:

- [ERRPIDR1.DES_0](#): JEP106 identification code bits[3:0].
- [ERRPIDR2.DES_1](#): JEP106 identification code bits[6:4].
- [ERRPIDR4.DES_2](#): JEP106 continuation code.

These codes indicate the designer of the component and not the implementer, except where the two are the same. To obtain a number, or to see the assignment of these codes, contact JEDEC <http://www.jedec.org>.

A JEP106 identification and continuation code takes the following form:

- A sequence of zero or more numbers, all having the value $0 \times 7F$.
- A following 8-bit number, that is not $0 \times 7F$, and where bit[7] is an odd parity bit.

The parity bit in the JEP106 identification code is not included.

This field has an IMPLEMENTATION DEFINED value.

Note

For example, Arm Limited is assigned the code $0 \times 7F \ 0 \times 7F \ 0 \times 7F \ 0 \times 7F \ 0 \times 3B$.

- The continuation code is the number of times $0 \times 7F$ appears before the final number. For example, a component designed by Arm Limited has the code 0×4 .
- The identification code is bits[6:0] of the final number. For example, a component designed by Arm Limited has the code $0 \times 3B$.

Access to this field is RO.

Accessing ERRPIDR4

This section shows the offset of ERRPIDR4 when FEAT_RASSA_4KB_GRP is implemented. If FEAT_RASSA_16KB_GRP or FEAT_RASSA_64KB_GRP is implemented, see ‘RAS memory-mapped register views’ for the offset of ERRPIDR4.

ERRPIDR4 can be accessed through the memory-mapped interface:

Component	Offset	Instance
RAS	$0 \times FD0$	ERRPIDR4

Accesses to this register are RO.

Glossary

Availability	Readiness for correct service.
Baseboard Management Controller	A PE dedicated to system control and monitoring.
BIST	Built-in self-test.
Built-in self-test	A mechanism that permits a machine to test itself.
Catastrophic failure	A failure with harmful consequences that are orders of magnitude, or even incommensurably, higher than the benefit provided by correct service delivery.
CE	Can refer to either a <i>corrected error</i> or the <i>Corrected</i> component error state..
Completer	An agent in a computing system that responds to and completes a transaction initiated by a Requester.
Component error state	A syndrome value recorded when a RAS node records that an error has been detected, that informs software of what actions the component has taken, and directs software recovery.
Contained or containable error	An error that is not uncontained or uncontainable.
Containment	Limiting or preventing the silent propagation of an error. Arm recommends that the scope to which an error is contained is specified.
Corrected error	An error that is detected by hardware and that hardware has corrected.
DE	Can refer to either a <i>deferred error</i> or the <i>Deferred</i> component error state.
DECTED	Double error correct, triple error detect EDAC. This can detect a single, double or triple bit error and correct a single or double bit error in a protection granule.
Deferred error	An error that has not been silently propagated but does not require immediate action at the producer. The error might have passed from the producer to a consumer.
Detected error	An error that has been detected and signaled to a consumer.
Detected Uncorrected Error	A detected error that has not been corrected and causes failure.
Device memory	Memory locations where an access to the location can cause side-effects, or where the value returned for a load can vary depending on the number of loads performed. Typically, the Device memory attributes are used for memory-mapped peripherals and similar locations.

DUE	Detected Uncorrected Error.
DUE FIT rate	The FIT rate for failures from a DUE.
ECC	Error Correction Code.
EDAC	Error Detection and Correction Code.
EDC	Error Detection Code.
Error	Deviation from correct service or a correct value.
Error Correction Code or Error Detection and Correction Code	A code capable of detecting and correcting a number of errors.
Error Detection Code	A code capable of detecting, but not correcting, errors.
Error log	Historical data recorded about errors, usually by software.
Error propagation	Passing an error from a producer to a consumer.
Error record	Data recorded about an error, usually by hardware.
Exception	An exception handles an event. For example, an exception could handle an external interrupt or an undefined instruction.
External abort	Either: - An in-band error that is generated as a response to a transaction. The name derives from the specific case of an abort generated by a memory system that is external to a PE, but the concept can apply to other interfaces. - A type of exception in the Arm architecture, generated when consuming an in-band error response.
Fail-safe	A failure mode in which the PE and other system components switch to backup mechanisms that keep processing instructions and data to allow either a safe shutdown or restart of the system, or to continue processing critical functions, or both.
Fail-secure	A failure mode in which the PE and other system components fail but the system is secured to allow either a safe shutdown or restart of the system, or to continue processing critical functions without exposing secret data, or both.
Fail-signaled	A failure mode in which the PE signals to the system that it has failed. It might continue to process instructions, but the system must ignore its output, or treat all outputs as detected errors.
Fail-silent	Failure mode in which the PE and all other system components (such as DMAs) stop processing instructions. A watchdog process will detect the failure and restart the system with an Error Recovery reset.
Failure	The event of deviation from correct service.
Failure-in-Time	The number of expected failures per billion hours of operation.
Fault	The cause of an error.
Fault injection	The deliberate injection of faults into a system for testing.
Fault prevention	Designing a system to avoid faults.
Fault removal	Logic or other mechanisms for detecting faults and correcting or bypassing their effect.
Field Replaceable Unit	A component or unit in a system that can be replaced without return to base.
FIT	Failure-in-Time.
FRU	Field Replaceable Unit.
Generic Interrupt Controller	Arm system architecture interrupt controller for IRQ and FIQ interrupt exceptions.

GIC	Generic Interrupt Controller.
Hardware fault	A fault that originates in, or affects, hardware.
Infected	Being in error.
Interrupt	An asynchronous event sent to a PE or GIC for processing as an interrupt exception.
Isolation	Limiting the impact of an error only to components that actually try to use corrupted data.
Latent error or latent fault	An error that is present in a system but not yet detected.
MBIST	Memory BIST.
Minor failure	A failure with harmful consequences that are of a similar cost to the benefits that are provided by correct service delivery.
MSI	Message Signaled Interrupt.
Normal memory	Used for bulk memory operations. Hardware might speculatively read these locations.
PCIe	Peripheral Component Interconnect Express.
PE	Processing Element.
Peripheral Component Interconnect Express (PCI Express or PCIe)	A high-speed serial computer expansion bus standard maintained and developed by the PCI Special Interest Group.
Persistent fault	A fault that is not transient.
PFA	Predictive Failure Analysis.
Poisoned	State that has been marked as being in error so that subsequent consumption of the state will be treated as a detected error.
PPI	Private Peripheral Interrupt.
Predictive Failure Analysis	Mechanisms to analyze errors and predict future failures.
Processing Element (PE)	The abstract machine defined in the Armv8 architecture, as documented in an Arm Architecture Reference Manual. A PE implementation compliant with the Armv8 architecture conforms with the behaviors described in the corresponding Arm Architecture Reference Manual.
Propagated	See Error propagation.
Protection granule	A quantum of memory for which an EDC or ECC provides detection or correction. For example, a 72/64 SECDED ECC scheme has a 64-bit protection granule.
RAS	Reliability, Availability, Serviceability.
Recoverable state	The state of a component when it has contained an error, and that error must be corrected to allow the correct operation of the system or smaller parts of the system to continue.
Reliability	Continuity of correct service.
Requester	An agent in a computing system that initiates transactions.
Restartable state	The state of a component when it has contained error, and the error does not immediately impact correct operation. Usually this means correct operation of the system, but it can also be used in other contexts to describe correct operation of a smaller part.
SDC	Silent Data Corruption.
SDC FIT rate	The FIT rate for failures because of SDC.
SDEC	Single device error correction EDAC. This can detect and correct multiple clustered errors in a protection granule, such as the types of errors that might be seen if a protection granule is striped across multiple devices and multiple errors come from a single device.

- SEDED** Single error correct, double error detect EDAC. This can detect a single or double bit error and correct a single bit error in a protection granule.
- SED** Single error detect EDC. This can detect a single bit error in a protection granule.
- Service failure mode** A mode entered to reduce the severity of an error.
- Serviceability** The ability to undergo modifications and repairs.
- Silent Data Corruption** An error that is not detected by hardware or software.
- Silently propagated** An error that is passed from place to place without being signaled as a detected error.
- Software fault** A fault that originates in and affects software.
- System Control Processor** A PE dedicated to system control and monitoring.
- Transient fault** A fault that is not persistent.
- UE** Can refer to either an *uncorrected error* or the *Uncorrected* component error state.
- Uncontained or uncontainable error** An error that has been, or might have been, silently propagated.
- Undetected error or undetected fault** See Latent error or latent fault.
- Unrecoverable state** The state of a component that has contained an error, but the state is not recoverable and continued correct operation is generally not possible. Usually this means correct operation of the system, but it can also be used in other contexts to describe correct operation of a smaller part. Systems might use high-level recovery techniques to work around an unrecoverable yet contained error in a component so that the system recovers from the error.